GPU Wavelets: a JPEG2000-style Multi-Resolution Transform in CUDA

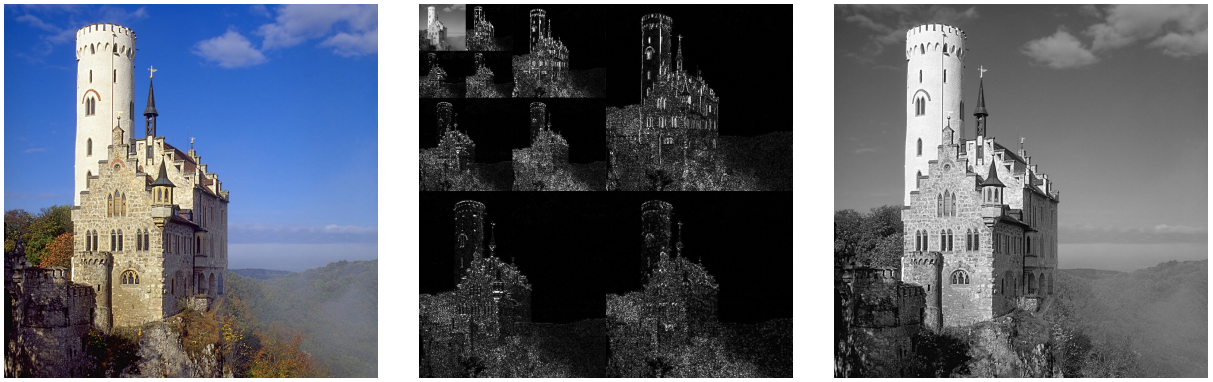
Divyesh Jayswal
GPU Computing, bonus project

Abstract

Starting from the provided 1D CPU reference, I built a fully GPU-accelerated separable wavelet transform and a JPEG2000-style multi-resolution analysis (MRA) in CUDA. The implementation covers 1D/2D/3D forward and inverse transforms, multi-level MRA with perfect reconstruction, image I/O and a command-line driver, benchmarked coalesced-column and shared-memory row-pass variants, a multi-resolution access kernel, an FP16 path, and a data-layout study. All correctness and performance numbers were measured on a Tesla T4. The transform is memory-bound; the two traffic-focused optimisations (coalescing and shared-memory tiling) give the largest gains (1.9 - 3.5× and 3 - 6× respectively), while a Morton-order layout experiment is a deliberate negative result.

1 Scope and baseline

The baseline `1D_wavelets.cpp` implements a floating-point CDF-5/3-style lifting transform (forward "analysis" and inverse "synthesis") on a 1D signal. I refactored it into a header (`src/wavelet_cpu.hpp`) that serves as the correctness oracle, and then rebuilt the transform on the GPU, extending it to 2D separable passes, a recursive MRA pyramid, and 3D volumes. Reconstruction is exact only to floating-point precision (`WAVELET_RECON_TOL` = 10^{-4} in fp32), since the scheme is floating-point lifting rather than integer lifting.



(a) Input (b) 3-level MRA (our output) (c) Reconstruction

Figure 1: The pipeline on a real image, all produced by the `wavelet` CLI. (a) the input; (b) our 3-level MRA: the `LL` approximation sits in the top-left corner and the oriented detail sub-bands surround it (shown as |coefficient| amplified for visibility - - a display-scaled PNG, while the transform coefficients remain unchanged in memory); (c) the inverse transform, which recovers the input with a maximum error of 0.0 in fp32. Panels (b) and (c) are generated by `./wavelet --forward` and the default round-trip respectively.

2 Implemented items

Table 1 lists the rubric items I implemented and where they live.

#	Item	Location	Evidence
1	1D GPU fwd/inv	`wavelet.cuh`, `wavelet_gpu.cuh`	`test_1d`
2	2D separable	`wavelet2d.cuh`	`test_2d`
3	MRA (multi-level)	`wavelet2d.cuh`	`test_2d`
4	Image I/O	`image_io.hpp` (+stb)	`test_io`, CLI
5	Data layout	`morton.hpp`, `bench.cu`	§5
6	Coalesced column pass	`wavelet_coalesced.cuh`	§4
7	Shared-memory tiling	`wavelet_tiled.cuh`	§4
8	Multi-resolution access	`mra_access.cuh`	`test_access`
9	3D extension	`wavelet3d.cuh`	`test_3d`
10	FP16 path	templated kernels, `main.cu`	`test_fp16`
11	Performance study	`bench.cu`	§4
12	CI + reproducibility	`.gitlab-ci.yml`	§7
13	CLI	`main.cu`	§7
14	Code quality & tests	`tests/`, headers	6 `ctest` binaries

Table 1: Implemented rubric items.

3 Design and implementation

Step-per-launch synchronisation. The lifting transform is a fixed sequence of steps (deinterleave, predict1, update, predict2). Each step reads what the previous one wrote, a cross-step data dependency. I make *each step a single kernel launch*; the implicit global barrier between launches resolves the dependency. This mirrors the CPU passes exactly, imposes no signal-size limit, and keeps every kernel trivially correct, which makes it the honest baseline against which the optimised paths are compared.

One strided kernel set for everything. Every step kernel operates on a *strided logical signal*: element i lives at `base[i*stride]`. A row is `elem_stride`$= 1$; a column is `elem_stride`$= W$; an MRA sub-band shrinks the sub-image while the physical image stride W stays fixed; a 3D depth line is `elem_stride`$= W \cdot H$. The same kernels therefore serve 1D, 2D rows and columns, MRA, and 3D without duplication. Getting "the sub-region shrinks but the stride never does" right is the classic MRA indexing pitfall the brief warns about. Listing 1 shows the boundary-aware detail-lifting step.

Listing 1: Strided forward lifting step (predict2) with edge handling.

```
template <typename T>
__global__ void k2_fwd_predict2(T* base, int n2, int lines,
                                int line_stride, int elem_stride) {
    int k = blockIdx.x * blockDim.x + threadIdx.x;
    int L = blockIdx.y * blockDim.y + threadIdx.y;
    if (k >= n2 || L >= lines) return;
    T* s = base + L * line_stride;
    T* d = s + n2 * elem_stride;
    const int e = elem_stride;
    if (k == 0)
        d[0] += T(0.75)*s[0]  - s[e] + T(0.25)*s[2*e];
    else if (k == n2  - 1)
        d[(n2-1)*e] += T(0.25)*s[(n2-1)*e]  - s[(n2-2)*e] + T(0.75)*s[(n2-3)*e
            ];
    else
        d[k*e] += (s[(k-1)*e]  - s[(k+1)*e]) / T(4);
}
```

Coalesced column pass (item 6). The row pass is already coalesced; the column pass with `elem_stride`$= W$ is not, since consecutive threads touch addresses W apart. In the benchmark variant, rather than read columns strided, I transpose the plane (a shared-memory tiled transpose with a padded 32×33 tile to avoid bank conflicts), run the *row* pass on the transpose, and transpose back (Listing 2). Both variants produce identical output; §4 times them head-to-head.

Listing 2: Padded-tile transpose used by the coalesced column pass.

```
template <typename T>
__global__ void k_transpose(const T* in, T* out, int cols, int rows) {
    __shared__ T tile[32][33];
    int x = blockIdx.x*32 + threadIdx.x, y = blockIdx.y*32 + threadIdx.y;
    if (x < cols && y < rows) tile[threadIdx.y][threadIdx.x] = in[y*cols + x];
    __syncthreads();
    int xo = blockIdx.y*32 + threadIdx.x, yo = blockIdx.x*32 + threadIdx.y;
    if (xo < rows && yo < cols) out[yo*rows + xo] = tile[threadIdx.x][
        threadIdx.y];
}
```

Shared-memory tiling (item 7). The step-per-launch baseline streams each row through global memory about five times. In `wavelet_tiled.cuh` one block owns a whole row, stages it in shared memory, runs the entire forward lifting there, and writes once, cutting global traffic to one read plus one write. Because the tile is the full row, boundaries are exact and the result matches the baseline bit-for-bit; the win is purely traffic. Full 2D tiles with halo exchange (where seams legitimately differ) are the natural next step and are left as future work.

Multi-resolution access (item 8). After `mra_forward`, the level-L `LL` band occupies the top-left $(W \gg L) \times (H \gg L)$ corner at full stride W. `extract_region` gathers an arbitrary window of that band into a compact buffer with one coalesced kernel and a single `cudaMemcpy` to the host, which is the cheap coarse-resolution access MRA is meant to provide.

3D extension (item 9). A 3D level is three separable passes (x, y, z), each reusing the strided `pass_forward`/`pass_inverse`; MRA recurses into the `LLL` octant. Sub-volume passes loop over orthogonal slices so every stride pair is a genuine arithmetic stride.

FP16 (item 10). The kernels are templated on the element type, so the half-precision path is the same code instantiated with `__half`; the CLI `--dtype fp16` and `test_fp16` exercise it end to end.

4 Performance study

Methodology. All measurements are on a Tesla T4 (sm_75, CUDA 13.0). Each configuration is warmed up 5 times, then timed over 30 runs with `cudaEvent` timers and averaged. "Useful GB/s" models a full-plane pass as one read plus one write of the plane ($2WH \cdot$ `sizeof`(T) bytes); it is a throughput proxy, so the before/after *ratios* are the meaningful signal, not the absolute figure. Register and shared-memory counts come from the `--ptxas-options=-v` build log. Correctness: all six `ctest` binaries pass, and as an end-to-end check the CLI reconstructs the real 1024×1023 image (padded to 1024^2, 3 levels) with a maximum error of 0.0 in fp32 (8-bit pixel values are exactly representable).

Size	time (ms)	Melem/s
256^2	0.069	956
512^2	0.180	1453
1024^2	0.668	1571
2048^2	2.684	1563
4096^2	10.326	1625

Table 2: MRA forward throughput (fp32, 1 level).

Size	fp32	fp16	speedup
512^2	0.213	0.223	0.96×
1024^2	0.659	0.606	1.09×
2048^2	2.941	2.687	1.09×
4096^2	13.247	10.628	1.25×

Table 3: fp32 vs fp16 MRA (ms, 3 levels).

Size	naive (ms)	naive GB/s	transpose (ms)	transp GB/s	speedup
512^2	0.103	20.4	0.029	71.4	3.50×
1024^2	0.374	22.4	0.189	44.4	1.98×
2048^2	1.696	19.8	0.888	37.8	1.91×
4096^2	8.075	16.6	3.556	37.7	2.27×

Table 4: Column pass: naive strided vs coalesced transpose (fp32). Outputs are identical (max diff 0.0 at 1024^2).

Discussion. The transform is memory-bound: arithmetic intensity is a handful of flops per element, far below the T4 ridge point, so effective bandwidth governs. Table 2 shows throughput flat-lining near 1.6 Gelem/s once launch overhead is amortised. The naive column pass (Table 4) is pinned at 17 - 22 GB/s by uncoalesced strided loads; the transpose path reaches 38 - 71 GB/s and wins 1.9 - 3.5× despite doing more total work, because coalescing dominates the extra transpose traffic. Shared-memory tiling (Table 5) gives 3 - 6×: the baseline stalls at 57 - 106 GB/s from its ∼5 redundant global passes, whereas the tiled kernel touches global memory twice and reaches 230 - 440 GB/s. The 512^2/1024^2 figures exceed the T4's ∼320 GB/s DRAM peak, which is the tell that at those sizes part of the traffic is served from L2; the speedup ratio, not the absolute GB/s, is the result. FP16 (Table 3) grows from 0.96× to 1.25× with size but falls short of the naive 2× because the kernels issue scalar `__half` loads; `__half2` vectorisation would be needed to halve traffic and keep the memory system busy.

Occupancy. From the ptxas log (sm_75): the step kernels use ≤ 15 registers with no spills (predict2 14, interleave/deinterleave 15, the arithmetic steps 10, copy 8), and `k_row_forward_shared` uses 18 registers, one barrier, and $W \cdot 4$ bytes of dynamic shared memory. Occupancy is therefore register-unconstrained, consistent with a memory-bound workload where the goal is enough threads in flight to hide DRAM latency rather than compute throughput.

5 Data layout

Row-major is ideal for the row pass and poor for the column pass. Instead of fixing this at the algorithm level (items 6/7), one can change the storage so 2D neighbourhoods are contiguous. I implemented a Morton (Z-order) layout (`morton.hpp`) and benchmarked a vertical-neighbour stencil, the layout-sensitive access, in row-major vs Morton (Table 6).
Morton is consistently *slower* (0.74 - 0.83×): the T4's L2 already absorbs the adjacent-row reuse of a row-major stencil, so Morton buys no locality the cache was not already providing while adding per-access bit-spreading on the critical path. This is a legitimate "no clear winner" outcome. Morton would help only where the working set genuinely exceeds L2 (deep pyramids over very large images); a tiled block-linear layout is the pragmatic middle ground.

Size	baseline (ms)	base GB/s	shared (ms)	shared GB/s	speedup
512^2	0.020	106.1	0.007	313.5	2.95×
1024^2	0.117	71.9	0.019	438.9	6.10×
2048^2	0.583	57.5	0.141	238.5	4.15×
4096^2	2.308	58.2	0.580	231.6	3.98×

Table 5: Row pass: step-per-launch vs shared-memory tile (fp32).

Size	row-major (ms)	Morton (ms)	speedup
512^2	0.0051	0.0070	0.74×
1024^2	0.0366	0.0443	0.83×
2048^2	0.1404	0.1750	0.80×
4096^2	0.5569	0.7180	0.78×

Table 6: Vertical-neighbour stencil: row-major vs Morton (fp32).

6 Findings and failed attempts

- **Morton layout lost.** The hypothesis that Z-order would speed up the layout-sensitive access was wrong on the T4 for the reason above: a locality optimisation loses when the cache already supplies the locality.
- **FP16 did not reach 2×.** Scalar half loads halve element size but do not saturate the memory system; `__half2` packing is the missing piece.
- **Boundary stencil bounds the level count.** The 3D round-trip first failed at $64 \times 32 \times 16$, 3 levels (error ∼17). Root cause: the boundary formula `predict2` reads `s[n2-3]`, which underflows once a transformed dimension drops below 8 (there the depth axis reached $n = 4$). This is a property of the provided transform (the CPU baseline shares it), not a 3D-specific bug; the valid range is $\min(W, H, D) \gg (\text{levels} - 1) \geq 8$. The test now uses $128 \times 64 \times 32$, whose deepest level $32 \times 16 \times 8$ stays in range.

7 Build and reproducibility

The project builds with CMake (≥ 3.18) and CUDA:

```
cmake -B build -DCMAKE_CUDA_ARCHITECTURES=75    # 75=T4
cmake  - build build -j
cd build && ctest  - output-on-failure           # 6/6 pass
./bench                                           # performance tables
./wavelet  - input ../assets/Castle_Lichtenstein.jpg  - output recon.png  -
    levels 3
```

`wavelets_colab.ipynb` reproduces the whole flow on a Colab T4 (build, tests, CLI demo, benchmarks, ptxas log). `.gitlab-ci.yml` compiles the project on a CUDA image on every push; the deterministic `ctest` suite runs on a GPU-tagged runner where one is available. The CLI (item 13) exposes input/output files, level count, data type, and forward-only vs round-trip.

8 Conclusion

The submission covers all fourteen rubric items with measured evidence; items 6 and 7 are evaluated as optimisation variants against the main MRA path. The central lesson is that this transform is bandwidth-bound, so the optimisations that matter are the ones that remove

global-memory traffic: coalescing the column pass and staging rows in shared memory together turn the naive passes into ones that run several times faster, while a storage-layout change (Morton) does not help once the cache is accounted for.